

mem0 Vector Store Benchmark Report — Aerospike vs Redis vs Postgres

Date: 2026-10-01 **Runner:** ai-ecosystem-benchmark 0.0.6 (same driver as burr-aerospike load tests) **Workloads:** mem0_aerospike_vector_workload.py, mem0_redis_vector_workload.py, mem0_postgres_vector_workload.py **Environment:** Apple Silicon Mac, Docker Desktop — **Aerospike 8.x server runs under x86_64 emulation**; Redis and Postgres run native ARM. Single-node container per backend. **Backends:** mem0.vector_stores.aerospike.AerospikeDB (preview aerospike-sdk + aerospike-async), mem0.vector_stores.redis.RedisDB (Redisearch FLAT), mem0.vector_stores.pgvector.PGVector (pgvector, exact, hnsw=False diskann=False, maxconn=64).

Methodology

All three backends execute the **same five VectorStoreBase scenarios** — get, insert, update, search (cosine, filtered by user_id), list (filtered by user_id) — driven by the same AI-agent-style pacer.

Fixed parameters across all runs:

Parameter	Value
Embedding dims	1536
Payload size	256 B
Top-K	10
Target QPS	500
Worker cap	256
Warmup	1 s
Duration	10 s per scenario (5,000 calls)
Seed	20260923

Sweep axis: corpus_size × key_pool_size, which controls the **filtered candidate set** per query (corpus ÷ pool):

Corpus / Pool	Candidates per user_id
1,000 / 50	20
10,000 / 50	200
10,000 / 5	2,000
100,000 / 50	2,000 (at 10× total size)

Latency below is **service** latency (execution-only, excludes client queue wait) unless noted. All runs: 5,000 attempted calls, 0 failures.

Table 1 — search service latency (ms) across candidate-set sweep, 500 qps / 256 workers

Backend	1k/50 (20 cand)	10k/50 (200 cand)	10k/5 (2k cand)	100k/50 (2k cand)
Aerospike	2.06 / 4.10 / 50.86	2.06 / 2.42 / 4.13	4.06 / 4.26 / 4.39	4.13 / 4.26 / 6.42
Redis	2.46 / 4.33 / 6.29	2.72 / 6.09 / 24.38	241.17 / 253.75 / 262.14	205.52 / 226.49 / 262.14
Postgres	8.06 / 14.16 / 19.92	11.53 / 21.23 / 34.08	59.77 / 96.47 / 117.44	1157.63 / 2046.82 / 2248.15

Cells are p50 / p95 / p99.

Achieved qps where the target was not met: Redis 399 (10k/5), 475 (100k/50); Postgres 205 (100k/50). Aerospike achieved the full ~500 qps in every cell.

Table 2 — All scenarios, 10k/50 (200 cand), 500 qps / 256 workers

Service latency p50 / p95 / p99 (ms). Note AerospikeDB.list() uses limit(top_k) — unordered, matching get_all semantics:

Scenario	Aerospike	Redis	Postgres
get	0.50 / 2.02 / 2.02	0.47 / 2.10 / 2.23	2.36 / 2.59 / 6.03
insert	0.56 / 2.03 / 2.08	2.02 / 2.13 / 2.42	17.83 / 34.60 / 43.52
update	0.56 / 2.03 / 2.08	2.02 / 536.87 / 553.65	12.19 / 23.86 / 30.41
search	2.06 / 2.42 / 4.13	2.72 / 6.09 / 24.38	11.53 / 21.23 / 34.08
list	2.05 / 2.13 / 2.52	165.68 / 205.52 / 285.21	2.36 / 4.03 / 4.59

Caveats

- **Emulation penalty:** Aerospike server 8.x has no ARM image; it runs under QEMU/Rosetta x86_64 emulation while Redis and Postgres are native. Point-op parity despite emulation suggests the query-path gaps are real, but absolute magnitudes may tighten on native x86 hardware.
- **Single node, Docker Desktop:** not a capacity study — a cross-sectional comparison of integration-shaped workloads.
- **pgvector runs exact** (hnsw=False, diskann=False) and stores user_id in JSONB without a dedicated index, matching the out-of-box mem0 config. Enabling HNSW or indexing payload->>'user_id' would change the Postgres cells materially.
- **Aerospike uses the preview Python SDK** (aerospike-sdk + aerospike-async, server 8.1.3+) with secondary-index + expression-engine top-K; results reflect the current exact-search engine, not a final GA implementation.

- **Run-to-run variance is significant** on this shared Docker Desktop host: e.g., Redis update at 10k/50 spiked to 537 ms p95 this sweep (achieved 330 qps) after measuring ~2 ms in prior runs, and earlier Aerospike list/search runs showed multi-second p95s that were not reproducible. Treat single-cell outliers skeptically; raw JSONs for every run are kept alongside this report.

Reproduce

```
cd mem0-aerospike/benchmarks
```

```
docker compose up -d --wait aerospike redis-valkey postgres
```

```
# Aerospike (emulated container on :3100)
```

```
python -m benchmarks.mem0_aerospike_vector_workload \  
  --aerospike-connection-string localhost:3100:test \  
  --corpus-size 100000 --key-pool-size 50 --qps 500 --worker-threads 256 \  
  --output benchmark_results/aerospike-100000-50.json
```

```
# Redis / Postgres – same flags, different connection args
```

```
python -m benchmarks.mem0_redis_vector_workload --redis-url redis://localhost:  
6379 ...
```

```
python -m benchmarks.mem0_postgres_vector_workload \  
  --postgres-connection-string postgresql://postgres:postgres@localhost:5432/  
postgres ...
```

```
Raw JSON: benchmark_results/{aerospike,redis,postgres}-{corpus}-{pool}.json.
```